

01-current-state-bottleneck-audit

HelixCode Speed Programme — R1: Current-State Bottleneck Audit

Revision	1
Created	2026-05-20
Last modified	2026-05-20T10:36:08Z
Status	active
Authority	docs/research/speed/ — operator speed mandate 2026-05-20

Table of contents

- [1. Executive summary](#)
- [2. Method & scope](#)
- [3. Ranked bottleneck table](#)
- [4. Per-bottleneck detail](#)
- [5. Cross-cutting observations](#)
- [6. Measurement gaps](#)
- [7. Recommended next research](#)

1. Executive summary

This audit reads the real HelixCode inner Go module (helix_code/, module dev.helix.code, go 1.26, ~45 internal packages) to inventory where the product is — or will be — slow versus competitor AI CLI agents (Claude Code, Gemini CLI, Aider, Cline). Every claim below cites a real file:line or a measured fact; speculative items are marked UNCONFIRMED:.

Top findings (highest-impact first):

1. **CLI startup is fully eager and serial.** `cmd/cli/main.go Run()` constructs ~20 subsystems sequentially before the first user action: telemetry, permissions, persistence, worktree (shells out to git), hooks (2 YAML reads), tool registry, LSP detection ($N \times \text{exec.LookPath}$), MCP manager (spawns servers), background manager, plan-mode, sandbox detection. Nothing is lazy. This is the single biggest perceived-latency gap vs competitors that defer almost everything.
2. **NewOllamaProvider does a synchronous network round-trip in its constructor** (`internal/llm/ollama_provider.go:109` → `discoverModels()`), and `NewCLI()` calls it unconditionally (`cmd/cli/main.go:234`). Every CLI invocation pays an Ollama discovery call even for non-LLM commands.
3. **Cloud LLM providers use untuned `http.Clients`.** 8 of 9 providers construct `&http.Client{Timeout: ...}` with the *default* `http.Transport` (no `MaxIdleConnsPerHost`, no `ForceAttemptHTTP2`). Only Groq tunes it (`groq_provider.go:151`). Default `MaxIdleConnsPerHost` is 2 — connection churn on every concurrent or rapid-fire request.
4. **Repomap is serial despite a `MaxConcurrency` config field that is never read.** `internal/repomap/repomap.go` walks files serially (`discoverFiles`, line 248), parses serially (`GetOptimalContext` loop, line 117), and `GetStatistics` (line 198) calls `extractFileSymbols` an extra full time per file. Tree-sitter parsing dominates and is single-threaded.
5. **Tree-sitter allocates a fresh parser per file** (`tree_sitter.go:56` `sitter.NewParser()` inside `ParseFile`) — no parser reuse or pooling across the thousands of files in a large repo walk.
6. **Repomap disk cache spawns one goroutine per Set**, each doing `MkdirAll` + `gob-encode` + `temp-write` + `rename` (`cache.go:101`). A cold index of a 5k-file repo spawns 5k short-lived goroutines hammering the filesystem.
7. **SearchContent (the grep tool) walks and reads files serially on one goroutine** (`internal/tools/filesystem/searcher.go:243`). Competitor `grep` backends (`ripgrep`) parallelise per-file aggressively.

None of these block working features; they are pure latency. The codebase compiles cleanly (`go build ./cmd/cli/` succeeds, 14.1 s warm) and has 113 benchmark functions, but coverage of the hot paths above is thin (see §6).

2. Method & scope

- Read the real code under `helix_code/cmd/` and `helix_code/internal/`. Confirmed `helix_code/` is the inner Go module (`go.mod` → module `dev.helix.code`, go 1.26). The CLAUDE.md text

describing a deeper helix_code/helix_code/ path is stale — the module is one level up.

- Verified compilation: go build ./cmd/cli/ → exit 0, **14.10 s** warm (captured via /usr/bin/time). go version go1.26.2 linux/amd64.
- Package-size mapping via find ... -name '*.go' ! -name '*_test.go': internal/tools 34,480 LOC / 131 files; internal/llm 30,564 LOC / 75 files; internal/memory 23,365 LOC / 26 files; internal/agent 7,601; internal/commands 7,250; internal/cognee 6,485; internal/config 4,182; internal/server 3,643; internal/context 3,363; internal/repomap 2,816.
- 113 func Benchmark* exist across the tree (grep -rh "func Benchmark").
- Could NOT run go test -bench against real infrastructure (no PostgreSQL/Redis/Ollama up in this session) — see §6.

3. Ranked bottleneck table

Severity: **Critical** = dominant user-visible latency; **High** = large, broadly hit; **Medium** = noticeable on hot paths; **Low** = localised.

#	Severity	Location
B01	Critical	cmd/cli/main.go Run() (lines 459–680+)
B02	Critical	internal/llm/ollama_provider.go:109; cmd/cli/main.go:23
B03	High	internal/llm/ {openai,deepseek,gemini,azure,copilot,cerebras,koboldai}_ _provider.go
B04	High	internal/repomap/repomap.go:117,184
B05	High	internal/repomap/repomap.go:218–235 (GetStatistics)
B06	High	internal/repomap/tree_sitter.go:56
B07	High	internal/repomap/cache.go:101,117,176
B08	High	internal/tools/filesystem/searcher.go:243 (SearchContent)
B09	Medium	internal/repomap/repomap.go:248; 18 other sites

#	Severity	Location
B10	Medium	cmd/cli/main.go:241 NewCLI() → config.Load()
B11	Medium	internal/llm/ollama_provider.go:354,393
B12	Medium	internal/editor/formats/*.go (≥17 sites)
B13	Medium	cmd/cli/main.go:572–573
B14	Medium	cmd/cli/main.go:589–605
B15	Medium	internal/repomap/cache.go:206,332 (GetStats/estimateSize)
B16	Medium	internal/repomap/repomap.go:103 GetOptimalContext
B17	Medium	internal/database/database.go:45-48
B18	Low	cmd/cli/main.go:301 initPersistence
B19	Low	internal/repomap/repomap.go:148
B20	Low	Makefile:35,49 test targets
B21	Low	internal/llm 30k LOC / 75 files single package
B22	Low (UNCONFIRMED)	internal/memory 23k LOC, 32 Lock() sites in memory_manag
B23	Low (UNCONFIRMED)	internal/cognee/service.go (32 lock sites)

4. Per-bottleneck detail

B01 — Eager, serial CLI bootstrap (Critical)

`CLI.Run()` (`cmd/cli/main.go`) parses flags at line 451 then, before dispatching to any handler, sequentially constructs: telemetry provider (476–504), permissions engine (510–514, reads 2 YAML files + resolves home/cwd), persistence manager (515–517), worktree manager (518–520, shells out to `git rev-parse --show-toplevel`), session manager (523), hooks (528–530, two YAML loads), tool registry (534, which itself initialises filesystem/shell/web/browser/mapper/multiedit components — `registry.go:201`), `ask_user` prompter (558), LSP curated-server detection (572–584), MCP manager load+start (589–605, spawns subprocesses), commands registry (614–618), background manager (622), plan-mode controller (634–648), sandbox config + detector (669–679). Each step is blocking. A user running `helixcode --list-models` still pays the worktree git shell-out, hooks YAML parse, LSP `exec.LookPath` sweep, and MCP server spawn. Competitor CLIs defer nearly all of this. **Fix:** restructure `Run()` so only flag parsing and the command-dispatch decision are eager; build each subsystem lazily inside the handler that needs it (`sync.Once-guarded` getters on `*CLI`).

B02 — Synchronous Ollama discovery in constructor (Critical)

`NewOllamaProvider` (`ollama_provider.go:99`) calls `provider.discoverModels()` at line 109 — a blocking HTTP GET to `localhost:11434/api/tags`. `NewCLI()` (`main.go:232`) calls `NewOllamaProvider` unconditionally for every CLI start. If Ollama is down, this still waits out the client `Timeout`. If Ollama is up, it's a wasted round-trip for non-LLM commands. **Fix:** construct the provider without discovery; discover lazily on first `GetModels/Generate`, or `async` in a goroutine whose result is awaited only when needed.

B03 — Untuned HTTP transports across providers (High)

Confirmed via `grep`: `openai_provider.go:46`, `deepseek_provider.go:50`, `gemini_provider.go:168`, `azure_provider.go:226`, `copilot_provider.go:45`, `cerebras_provider.go:66`, `koboldai_provider.go:86`, `local_provider.go:36`, `local_llm_manager.go:221` all build `&http.Client{Timeout: ...}` with no `Transport`. Go's default transport caps `MaxIdleConnsPerHost` at 2, so a burst of requests to one provider opens/closes TCP+TLS repeatedly. Only `groq_provider.go:151` sets `MaxIdleConnsPerHost: 100`

+ ForceAttemptHTTP2. **Fix:** a single shared, tuned *http.Transport (or per-provider with MaxIdleConnsPerHost ≥ 32, IdleConnTimeout, ForceAttemptHTTP2: true), reused across all providers.

B04 / B05 / B06 — Repomap parsing not parallel, double-parsed, parser churn (High)

RepoMapConfig.MaxConcurrency is declared (repomap.go:29, default 4 in DefaultConfig line 59) but grep finds no read of it — GetOptimalContext (line 117) and RefreshCache (line 184) loop over files serially. GetStatistics (line 218) loops over every file calling extractFileSymbols again even though the same files were just walked. TreeSitterParser.ParseFile allocates sitter.NewParser() per file (tree_sitter.go:56) — parsers are reusable and the allocation+language-set is non-trivial across thousands of files. **Fix:** bounded worker pool keyed off MaxConcurrency; a sync.Pool of parsers per language; single-pass stats.

B07 — Cache write goroutine storm (High)

RepoCache.Set (cache.go:85) spawns a goroutine per call (line 101) that does os.MkdirAll + gob.Encode + os.WriteFile(tmp) + os.Rename. Invalidate (117) and Cleanup (176) also spawn per-entry goroutines. Indexing a large repo cold → thousands of concurrent tiny FS ops, scheduler churn, and inode pressure. **Fix:** one background writer goroutine consuming a buffered channel of (key, entry); coalesce directory creation.

B08 — Serial content search (High)

fileSearcher.SearchContent (searcher.go:209) compiles the regex once (good, line 233) but then filepath.Walks and calls searchFileContent for each file inline on the walk goroutine (line 276). For a large tree this is strictly serial I/O + regex. Competitor grep tools parallelise per file. **Fix:** feed candidate paths into a bounded worker pool; collect matches via channel; this is the agent's Grep tool hot path.

B09 — filepath.Walk vs filepath.WalkDir (Medium)

19 filepath.Walk call sites found (repomap, context mentions, fix, rules, tools/mapping, tools/web, tools/filesystem, workflow, kilocode). Walk calls os.Lstat on every entry to build os.FileInfo; WalkDir passes a cheaper fs.DirEntry and only stats on demand. On large trees this is a measurable win. **Fix:** mechanical migration to filepath.WalkDir.

B10 — Repeated/global Viper config load (Medium)

`config.Load()` (`config/config.go:215`) uses the **global** Viper singleton (`viper.SetConfigFile`, `viper.ReadInConfig`, `viper.Unmarshal`). `NewCLI()` calls it at `main.go:241`; the server `main.go:75` calls it too; subagent path reads config files independently (`main.go:172`). Each `Load()` re-reads and re-unmarshals YAML. **Fix:** load once into a `*Config`, thread it through constructors; avoid the global singleton for testability and to stop repeat disk reads.

B11 — Redundant body copy in Ollama requests (Medium)

`makeAPIRequest/makeStreamingRequest` do `strings.NewReader(string(requestBody))` where `requestBody` is already `[]byte` from `json.Marshal` (`ollama_provider.go:359,398`). The `string()` conversion copies the whole payload. **Fix:** `bytes.NewReader(requestBody)`.

B12 — Per-call regexp compilation (Medium)

`internal/editor/formats/{editor,diff,architect,whole}_format.go` compile regexes inside parse functions (≥ 17 sites, e.g. `editor_format.go:53,63,120, 133,150,163`). These run on every LLM-edit-format parse. **Fix:** hoist to package-level `var ... = regexp.MustCompile(...)`.

B13 / B14 — Startup-time external probes (Medium)

`tools.DetectAvailableServers` (`main.go:573`) runs `exec.LookPath` for each curated LSP server synchronously. `mcpMgr.Start(ctx)` (`main.go:601`) launches alwaysLoad MCP server subprocesses during bootstrap. Both add fixed startup latency unrelated to the user's command. **Fix:** lazy detection on first edit / first MCP call; or at minimum parallelise the `LookPath` sweep.

B15-B23

See table; B15 (gob-encode-to-measure), B16 (over-broad RLock across I/O), B17 (non-configurable DB pool), B18 (unconditional cleanup goroutine), B19 (full read for token estimate), B20 (untuned test parallelism), B21 (30k-LOC `internal/llm` slows incremental compile), B22/B23 (UNCONFIRMED: lock contention in `memory/cognee` — 32 lock sites each; needs `-race` + contention profiling under concurrent load to confirm).

5. Cross-cutting observations

- **Lazy initialisation is the recurring theme.** B01, B02, B13, B14, B18 are all “work done eagerly that the current command may not need.” A single architectural pass introducing sync.Once-guarded lazy getters on *CLI would address the largest perceived-latency gap.
- **HTTP client hygiene is inconsistent.** Groq is the only provider with a tuned transport; the pattern should be lifted into a shared constructor.
- **Repomap is the clearest CPU bottleneck** and already has the scaffolding (MaxConcurrency, a cache) — it just isn’t wired for parallelism.
- **No evidence of broken correctness** in any hot path inspected; these are pure-throughput/latency issues. `go build ./cmd/cli/` is clean.
- **Build/incremental-compile:** warm `go build ./cmd/cli/` = 14.1 s; the 30k-LOC `internal/llm` and 34k-LOC `internal/tools` packages are large enough that any edit in them triggers a sizeable recompile (B21).

6. Measurement gaps

The following need real profiling that this audit could not perform and that R2+ of the speed programme must capture before/after each fix:

1. **Real startup wall-clock** — `time ./bin/cli --help` and `--list-models` on a representative machine; instrument `Run()` phase-by-phase. No binary was run in this session.
2. **pprof CPU + alloc profiles** for: a cold repomap index of a large repo (B04–B07), a content search over a large tree (B08), and a single LLM round-trip (B02/B03). None captured — needs Ollama + a real repo.
3. **go test -bench -benchmem** of the 113 existing benchmarks — not run here (needs `make test-infra-up`: PostgreSQL/Redis/Ollama). Confirm which hot paths (repomap, llm dispatch, searcher) actually have benchmark coverage; first inspection suggests repomap parsing and CLI startup have **no** dedicated benchmarks.
4. **Lock-contention profile** (`go test -race, runtime/pprof mutex profile`) for `internal/memory` and `internal/cognee` — required to confirm or dismiss B22/B23.
5. **Connection-reuse verification** — capture `netstat`/transport metrics to prove B03’s idle-conn churn under burst load.
6. **Competitor baseline** — measured startup + first-token latency for Claude Code / Gemini CLI / Aider / Cline on identical hardware, so the “3–5×” target has a concrete number to beat.

7. **Incremental compile timing** — go build after a one-line edit in internal/llm vs a small package, to size B21.

7. Recommended next research

- **R2:** Competitor latency baseline (startup, first-token, repo-index, grep) on identical hardware — gives the target numbers.
- **R3:** Profiling harness — wire pprof endpoints + a benchmark suite for the four hot paths (startup, llm dispatch, repomap, search) so every speed fix has before/after evidence (anti-bluff §11.4).
- **R4:** Lazy-init architecture design for cmd/cli (addresses B01/B02/ B13/B14 — the Critical tier).